

Go2 EDU: RealSense + Hesai Rosbag Recording Guide

Go2 EDU: RealSense (Humble container) + Hesai (Foxy native) Setup, Recording, and Calibration

Purpose

0. Prerequisites

1. SSH into the Jetson
2. Install Docker + NVIDIA Container Runtime
 - 2.1 Install Docker Engine
 - 2.2 Install NVIDIA Container Runtime (Jetson-specific)
3. Pull/Build a Humble Image for Jetson (arm64) and Install the RealSense Wrapper
 - 3.1 Pull a Jetson-compatible Humble base image
 - 3.2 Build a custom image with the RealSense wrapper installed
4. Run the RealSense Container
5. Configure the Foxy Host to Match
6. Install and Run the Hesai Driver (native Foxy)
7. Verify Both Streams on the Host
8. Record the Rosbag
9. Calibration: Aligning the RealSense Point Cloud to the Hesai Point Cloud
 - 9.1 Get a rough initial transform
 - 9.2 Capture a static calibration scene
 - 9.3 Extract point clouds for calibration
 - 9.4 Run ICP in Open3D
 - 9.5 Validate the result
 - 9.6 Freeze it as a static transform

Summary Checklist

Go2 EDU: RealSense (Humble container) + Hesai (Foxy native) Setup, Recording, and Calibration

Purpose

This guide's scope is narrow and deliberate: **get a proper, complete rosbag recorded on the Jetson, containing synchronized RealSense (RGB + depth) and Hesai lidar data**, ready to hand off for offline processing on a workstation. It does not cover SLAM, semantic mapping, SAM/CLIP fusion, or Nav2 — those are separate downstream steps that consume the rosbag this guide produces.

Target system: Go2 EDU onboard Jetson (JetPack 5.1.1, Ubuntu 20.04, ROS 2 Foxy pre-installed). Goal: run the actively-maintained RealSense ROS2 wrapper (which no longer supports Foxy) inside a Humble Docker container, bridge it to the Foxy host via DDS (no `ros1_bridge`), run the Hesai lidar driver natively on Foxy, verify both streams, and record a clean rosbag2 — plus a final section on calibrating the lidar-camera extrinsic, since that's needed to make sense of the recorded data afterward.

0. Prerequisites

- SSH access to the Jetson (default ssh unitree@192.168.123.18, password 123 unless changed)
 - Hesai lidar physically connected and network-configured per its manual
 - RealSense camera connected via USB to the Jetson
 - A workstation to pull the rosbag afterward for offline processing (not required for this doc)
-

1. SSH into the Jetson

```
ssh unitree@192.168.123.18
```

If the Jetson's clock reset to 1970 after a power cycle (known issue), connect it to WiFi/internet first so time syncs, or set it manually:

```
sudo date -s "YYYY-MM-DD HH:MM:SS"
```

Confirm you're on Foxy when you open a terminal (you may be prompted to choose ROS1 Noetic vs ROS2 Foxy — choose Foxy / option matching ROS2):

```
echo $ROS_DISTRO  
# should print: foxy
```

2. Install Docker + NVIDIA Container Runtime

2.1 Install Docker Engine

```
sudo apt update  
sudo apt install -y ca-certificates curl gnupg lsb-release  
  
sudo mkdir -p /etc/apt/keyrings  
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg -  
-dearmor -o /etc/apt/keyrings/docker.gpg  
  
echo \  
"deb [arch=$(dpkg --print-architecture) signed-  
by=/etc/apt/keyrings/docker.gpg]  
https://download.docker.com/linux/ubuntu \  
$(lsb_release -cs) stable" | sudo tee  
/etc/apt/sources.list.d/docker.list > /dev/null  
  
sudo apt update  
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-  
buildx-plugin docker-compose-plugin
```

Add your user to the docker group (avoids needing sudo for every docker command):

```
sudo usermod -aG docker $USER  
newgrp docker
```

Verify:

```
docker run hello-world
```

2.2 Install NVIDIA Container Runtime (Jetson-specific)

Most Jetson JetPack images already ship `nvidia-container-runtime`. Verify:

```
dpkg -l | grep nvidia-container
```

If missing:

```
sudo apt install -y nvidia-container-runtime
```

Set it as the default Docker runtime — edit `/etc/docker/daemon.json`:

```
sudo nano /etc/docker/daemon.json
```

```
{
  "runtimes": {
    "nvidia": {
      "path": "nvidia-container-runtime",
      "runtimeArgs": []
    }
  },
  "default-runtime": "nvidia"
}
```

Restart Docker:

```
sudo systemctl restart docker
```

Note: for this specific use case (RealSense capture only, no GPU inference happening inside this container) the NVIDIA runtime isn't strictly required. Set it up anyway if you plan to later run SAM/CLIP inference on-device — it's needed for that, not for RealSense capture itself.

3. Pull/Build a Humble Image for Jetson (arm64) and Install the RealSense Wrapper

3.1 Pull a Jetson-compatible Humble base image

```
docker pull dustynv/ros:humble-desktop-l4t-r35.1.0
```

Adjust the L4T tag to match your JetPack/L4T version if different (cat `/etc/nv_tegra_release` on the Jetson to check).

3.2 Build a custom image with the RealSense wrapper installed

Create a working directory and Dockerfile:

```
mkdir -p ~/realsense_humble_docker && cd ~/realsense_humble_docker
nano Dockerfile
```

```
FROM dustynv/ros:humble-desktop-l4t-r35.1.0
```

```
RUN apt update && apt install -y \
  ros-humble-realsense2-camera \
  ros-humble-realsense2-camera-msgs \
  ros-humble-rmw-cyclonedds-cpp \
  && rm -rf /var/lib/apt/lists/*
```

```
# Force CycloneDDS as the RMW implementation inside the container
ENV RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
```

```
CMD ["bash"]
```

If the apt package's librealsense version doesn't match your camera's firmware/features, build realsense-ros from source instead (ros2-development branch) inside the Dockerfile — apt is the faster path to start with.

Build it:

```
docker build -t go2-realsense-humble .
```

4. Run the RealSense Container

```
docker run -it --rm \
  --network host \
  --privileged \
  --device=/dev/bus/usb \
  -v /dev:/dev \
  -e RMW_IMPLEMENTATION=rmw_cyclonedds_cpp \
  -e ROS_DOMAIN_ID=0 \
  go2-realsense-humble
```

Key flags explained briefly: - --network host: required for DDS discovery to reach the Foxy host outside the container - --privileged / --device=/dev/bus/usb / -v /dev:/dev: USB camera passthrough - ROS_DOMAIN_ID: must match the value you use on the Foxy host (pick any consistent number, e.g. 0)

Inside the container, launch the camera with pointcloud and depth-color alignment enabled:

```
source /opt/ros/humble/setup.bash
ros2 launch realsense2_camera rs_launch.py \
  pointcloud.enable:=true \
  align_depth.enable:=true
```

Leave this running in its own terminal/tmux pane.

5. Configure the Foxy Host to Match

On the Jetson's **host** shell (outside the container), in every terminal you use for this project:

```
sudo apt install -y ros-foxy-rmw-cyclonedds-cpp # if not already
installed
source /opt/ros/foxy/setup.bash
export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
export ROS_DOMAIN_ID=0 # same value as
the container
```

Consider adding the two export lines to ~/.bashrc so you don't have to repeat them per session.

6. Install and Run the Hesai Driver (native Foxy)

```
mkdir -p ~/hesai_ws/src && cd ~/hesai_ws/src
git clone --recurse-submodules
https://github.com/HesaiTechnology/HesaiLidar_R0S_2.0.git

sudo apt install -y libboost-all-dev libyaml-cpp-dev

cd ~/hesai_ws
source /opt/ros/foxy/setup.bash
rosdep install --from-paths src --ignore-src -r -y
colcon build
source install/setup.bash
```

Edit the driver's config (under `src/HesaiLidar_R0S_2.0/config/`) to match your lidar model and network settings (`lidar_recv_port`, IP, `source_type` set to live UDP — not pcap/rosbag).

Launch:

```
ros2 launch hesai_ros_driver start.py
```

7. Verify Both Streams on the Host

With the RealSense container and Hesai driver both running, from the Foxy host:

```
ros2 topic list
```

You should see, among others:

```
/camera/color/image_raw
/camera/depth/image_rect_raw
/camera/depth/color/points
/hesai/pandar          # or /lidar_points depending on driver
version/config
```

Check data is actually flowing (not just topics registered):

```
ros2 topic hz /camera/depth/image_rect_raw
ros2 topic hz /hesai/pandar
```

Visual check with `rviz2` (run on the host or forward X11 / VNC in if headless):

```
rviz2
```

Add `PointCloud2` displays for both topics, set a common fixed frame, confirm both clouds appear sensibly (rough sanity check only — real alignment comes from calibration in Section 9).

If topics don't appear on the host:

```
ros2 daemon stop && ros2 daemon start
```

Then re-check `ros2 topic list`. Also confirm no firewall rule is blocking local multicast/UDP (shouldn't be an issue with `--network host` on the same machine, but worth a first check).

8. Record the Rosbag

Record all relevant topics into a single bag on the Jetson's local storage (use an SSD/fast storage if available — lidar + depth + RGB is bandwidth-heavy):

```
mkdir -p ~/bags && cd ~/bags
ros2 bag record \
  /camera/color/image_raw \
  /camera/depth/image_rect_raw \
  /camera/depth/color/points \
  /camera/color/camera_info \
  /camera/depth/camera_info \
  /hesai/pandar \
  /tf \
  /tf_static \
  -o go2_mapping_run_$(date +%Y%m%d_%H%M%S)
```

Add any odometry/pose topics your SLAM stack publishes (e.g. /odom, or whatever your lidar SLAM node outputs) so you have a trajectory to interpolate frame poses from later.

Recommendations: - Do **not** rely on live network streaming to a workstation during the actual mapping run — record locally, copy the bag off afterward (scp or pull the storage device). - Keep the robot's motion slow and deliberate during teleop — this benefits both SLAM loop closure quality and RGBD frame sharpness (motion blur hurts your later SAM/CLIP step). - After recording, copy the bag to your workstation: `bash scp -r unitree@192.168.123.18:~/bags/go2_mapping_run_XXXXXXXX_XXXXXX ~/local_bags/`

9. Calibration: Aligning the RealSense Point Cloud to the Hesai Point Cloud

This is a one-time rigid extrinsic calibration for the fixed sensor mount — not something you redo per frame or per session (unless the physical mount changes).

9.1 Get a rough initial transform

Measure (or pull from CAD/mount drawings) the approximate translation and rotation between the Hesai and RealSense mounting points. This doesn't need to be precise — it's just a starting guess so ICP converges to the correct local minimum instead of a wrong one.

9.2 Capture a static calibration scene

- Point both sensors at a feature-rich **static** area: a room corner, furniture edges, anything with clear 3D structure. Avoid a flat blank wall (under-constrains some axes).
- Keep the robot stationary.
- Capture at **close range (1-3 m)** — RealSense's depth range and FOV are much smaller than the Hesai's 360° sweep, so this keeps the sensor overlap region large enough for ICP to converge well.
- Record a short rosbag of both /hesai/pandar and the RealSense depth/pointcloud topics (`ros2 bag record` as in Section 8, just the relevant topics, for ~10-20 seconds).

9.3 Extract point clouds for calibration

On your workstation, replay the calibration bag and extract one (or an averaged/aggregated window of) point cloud message from each sensor — e.g. using `rosbag2_py` or simply echoing/saving a single `PointCloud2` message from each topic to a `.pcd` file via a small script using `open3d` or `sensor_msgs_py.point_cloud2`.

9.4 Run ICP in Open3D

```
import open3d as o3d
import numpy as np

source_realsense = o3d.io.read_point_cloud("realsense_frame.pcd")
target_hesai = o3d.io.read_point_cloud("hesai_frame.pcd")

# your rough initial transform from Section 9.1, as a 4x4 matrix
initial_guess = np.array([
    [1, 0, 0, 0.0],
    [0, 1, 0, 0.0],
    [0, 0, 1, 0.0],
    [0, 0, 0, 1.0],
])

source_realsense.estimate_normals()
target_hesai.estimate_normals()

result = o3d.pipelines.registration.registration_icp(
    source_realsense, target_hesai,
    max_correspondence_distance=0.05,
    init=initial_guess,

    estimation_method=o3d.pipelines.registration.TransformationEstimationPointToPlane
)

print(result.transformation)
```

Notes: - Point-to-plane ICP tends to converge better than point-to-point for structured indoor scenes. - If ICP struggles (sparse overlap, very different point densities), try Open3D's **colored ICP** (uses the RealSense RGB too) or run a global registration pass first (registration_fgr / RANSAC feature matching) before local ICP refinement. - Reduce max_correspondence_distance progressively across a couple of ICP passes for a tighter final fit once the coarse alignment looks right.

9.5 Validate the result

Transform both clouds into a common frame using `result.transformation` and visually inspect in Open3D or rviz2 from **multiple viewpoints** — a transform that looks correct head-on can still be off in depth. Check that edges/corners visually line up across both clouds.

9.6 Freeze it as a static transform

Once satisfied, extract translation + quaternion from the resulting 4x4 matrix and publish it as a permanent static TF:

```
ros2 run tf2_ros static_transform_publisher x y z qx qy qz qw
hesai_frame realsense_frame
```

If you maintain a URDF for the robot, bake this transform in as a fixed joint between the two sensor frames so it's automatically applied everywhere downstream (fusion pipeline, rviz, etc.) without needing the standalone static_transform_publisher node.

Summary Checklist

- ☐ Docker + NVIDIA container runtime installed on Jetson
- ☐ Humble image built with realsense2_camera installed
- ☐ RealSense container running with --network host, matching RMW_IMPLEMENTATION / ROS_DOMAIN_ID
- ☐ Foxy host env vars set to match (CycloneDDS, same domain ID)
- ☐ Hesai driver built and running natively on Foxy
- ☐ ros2 topic list / ros2 topic hz confirms both streams live on the host
- ☐ Rosbag recorded locally on Jetson during teleop mapping run, copied to workstation
- ☐ Static calibration scene captured, ICP extrinsic solved and validated
- ☐ Static transform published/baked into URDF between hesai_frame and realsense_frame